
Assignment Overview

You'll design a simple cloud infrastructure environment using Terraform (or OpenTofu) and Kubernetes.

We want to see your approach to cloud architecture, deployment practices, security thinking, and documentation.

You can find full task details at [the end of this document](#).

Deadline

Please submit your solution within **8 days** after receiving this email.

(If you need a little more time, feel free to let us know — we're happy to accommodate when needed.)

Tips

- Focus on clear, well-organized code and realistic infrastructure design.
 - You don't have to actually deploy into a live cloud environment if you prefer to just describe the deployment.
 - We value good documentation and practical decision-making over complexity.
-

How to Submit

Please reply to this email with:

- A link to a GitHub repo (preferred)
or
 - A zipped file containing your deliverables.
-

If you have any questions or need clarification about the assignment, feel free to reach out.
We're excited to see your work!

Best regards,

Flavio Castro

CIO

Antuar

flavio.castro@antuar.com

Take-Home Task: Cloud Infrastructure Engineer

Background

We run a containerized application across multiple cloud environments (AWS, Azure, GCP). We expect our infrastructure to be scalable, secure, and cost-efficient. Your task will simulate a small part of what you would work on with us.

Assignment

You are tasked with:

1 - Infrastructure-as-Code (IaC)

Write Terraform (or OpenTofu) code to provision the following on one of the three major clouds (AWS, Azure, or GCP — your choice):

- A Kubernetes cluster (managed service preferred: EKS, AKS, or GKE)
- A managed PostgreSQL database service

Requirements

Use modules or organize the code cleanly. Make sure the resources can communicate between them. Include a basic README explaining how to deploy it.

2 - Containerized Application

Application

Create a simple API server containerized application (you can use any language/framework you might be comfortable with but extra points if it is Java/Spring).

The application should have three endpoints:

1 - **POST** `/person` - Create/Save a person information

JSON payload:

attribute	Description
<code>email</code>	required , <i>unique</i> , String 100 chars
<code>name</code>	required , String 200 chars
<code>birthday</code>	required , String YYYY-MM-DD

attribute	Description
hobbies	optional, Array of Strings (60 chars each)

Example:

```
{
  "email" : "john.doe@gmail.com",
  "name" : "John Doe",
  "birthday" : "2000-10-01",
  "hobbies" : ["cars", "sports", "bowling"]
}
```

The application should validate the request and:

- Save the info in the database if the attributes are valid and return some **id** (identifier) for the person created and an OK status (201 status).
- return an error status with some detailed message if something is wrong.

2 - **GET /person/[:id]** - Retrieve details of a person

It should return a JSON with the details of a person that have the ID received as parameter with status 200. If the ID doesn't exist on the database, it should return a 404 status.

3 - **GET /**

It should show an HTML status page with some text describing the application name, etc., and the total number of persons added.

Deployment

Create Kubernetes manifests (or Helm charts) to:

- Deploy the app into the cluster
- Expose it publicly via a LoadBalancer Service

Ensure best practices like resource limits and basic readiness probes.

3. Cloud Security & Cost Optimization

In a short document (~1 page), answer the following:

Security

- Name 3 things you would do to secure this infrastructure if it were a production environment.
- Add any comments about security that might be relevant.

Cost

- Suggest 2 ways you would keep cloud costs under control at scale.
- Add any comments about costs that might be relevant.

4. (Optional Bonus) Monitoring

Optionally, configure the infrastructure to have a basic monitoring solution (built-in cloud monitoring OR attach a third-party like Prometheus) And create basic alerting/metrics collection for the deployed app (e.g., simple uptime checks, CPU alerts). Document what you set up.

Deliverables

- API Application code
- Terraform/OpenTofu code
- Kubernetes manifests (or Helm charts)
- README file explaining:
 - How to deploy the infrastructure
 - Any assumptions made
- Security & Cost Optimization document (short write-up)

Evaluation Criteria

- Code quality, clarity, and organization
- Infrastructure best practices
- Understanding of security and cost efficiency
- Simplicity and effectiveness of Kubernetes deployment
- Documentation and explanation skills

Notes

You don't actually have to deploy to your cloud account. You can describe assumptions if you prefer not to incur costs.

Focus on clean code, real-world sensibility, and explaining your thinking where necessary.